

From Euler to AI: Unifying Formulas for Mathematical Constants

Tomer Raz Michael Shalyt Elyashev Leibtag Rotem Kalisch
 Shachar Weinbaum Yaron Hadad Ido Kaminer*
 Technion – Israel Institute of Technology,
 Haifa 3200003, Israel

Abstract

The constant π has fascinated scholars throughout the centuries, inspiring numerous formulas for its evaluation, such as infinite sums and continued fractions. Despite their individual significance, many of the underlying connections among formulas remain unknown, missing unifying theories that could unveil deeper understanding. The absence of a unifying theory reflects a broader challenge across math and science: knowledge is typically accumulated through isolated discoveries, while deeper connections often remain hidden. In this work, we present an automated framework for the unification of mathematical formulas. Our system combines large language models (LLMs) for systematic formula harvesting, an LLM-code feedback loop for validation, and a novel symbolic algorithm for clustering and eventual unification. We demonstrate this methodology on the hallmark case of π , an ideal testing ground for symbolic unification. Applying this approach to 455,050 arXiv papers, we validate 385 distinct formulas for π and prove relations between 360 (94%) of them, of which 166 (43%) can be derived from a single mathematical object—linking canonical formulas by Euler, Gauss, Brouncker, and newer ones from algorithmic discoveries by the Ramanujan Machine. Our method generalizes to other constants, including e , $\zeta(3)$, and Catalan’s constant, demonstrating the potential of AI-assisted mathematics to uncover hidden structures and unify knowledge across domains.

1 Introduction

The earliest rigorous approximation for π dates back to Archimedes around 250 BCE, who established the bounds $\frac{223}{71} < \pi < \frac{22}{7}$ [16]. Modern π approximations employ more sophisticated formulas. For example, the Chudnovsky algorithm [15], derived from a formula by Ramanujan [48], remains instrumental for precision records. Similarly, the BBP formula [6] is notable for enabling computation of specific π digits without requiring prior digits. Such breakthroughs inspired fundamental advances in computer science, such as high-precision arithmetic [7], evolutionary optimization [35], and elliptic curve cryptography [39]. Recent efforts led to the development of computer algorithms capable of generating numerous formula hypotheses and sometimes proofs for mathematical constants [9, 17, 46].

*Corresponding author: kaminer@technion.ac.il
 Project repository: <https://github.com/RamanujanMachine/euler2ai>

2 Mathematical background

2.1 Recurrences as universal representations of formulas for mathematical constants

A wide range of formulas—including infinite sums, products, and continued fractions—can be converted into recurrences, providing a cohesive framework for unification. A function u_n satisfies a recurrence of order m if $u_n = a_{1,n}u_{n-1} + a_{2,n}u_{n-2} + \dots + a_{m,n}u_{n-m}$, which can be represented via the associated companion matrix:

$$\text{CM}(n) := \begin{pmatrix} 0 & 0 & \dots & 0 & a_{m,n} \\ 1 & 0 & \dots & 0 & a_{m-1,n} \\ 0 & 1 & \dots & 0 & a_{m-2,n} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & a_{1,n} \end{pmatrix} \quad (1)$$

By incrementally multiplying the companion matrix over n steps, we get the matrix:

$$\prod_{i=1}^n \text{CM}(i) = \begin{pmatrix} p_{1,n-m} & \dots & p_{1,n-1} & p_{1,n} \\ p_{2,n-m} & \dots & p_{2,n-1} & p_{2,n} \\ p_{3,n-m} & \dots & p_{3,n-1} & p_{3,n} \\ \vdots & \ddots & \vdots & \vdots \\ p_{m,n-m} & \dots & p_{m,n-1} & p_{m,n} \end{pmatrix} \quad (2)$$

$p_{1,n}, \dots, p_{m,n}$ are solutions to the recurrence for the initial conditions $p_{i,j} = \delta_i^j$. Other solutions for different initial conditions can be written as linear combinations of these.

Recurrences evaluate a desired constant L either directly $\lim_{n \rightarrow \infty} u_n = L$ (e.g., for infinite sums), or as ratios $\lim_{n \rightarrow \infty} \frac{p_n}{q_n} = L$ with p_n and q_n being two solutions for the recurrence (e.g., for continued fractions). In the special case of a second-order recurrence, $u_n = a_n u_{n-1} + b_n u_{n-2}$, and any pair of solutions is associated with a formula in the form of a continued fraction:

$$\frac{b_1}{a_1 + \frac{b_2}{\ddots + \frac{b_n}{a_n}}} = \frac{p_n}{q_n} \quad (3)$$

When the functions $a_n = a(n)$ and $b_n = b(n)$ are polynomials, the formula above is known as a Polynomial Continued Fraction (PCF), denoted by $\text{PCF}(a(n), b(n))$. See details in Appendix E.

2.2 The dynamical metrics describing each formula

A formula of a mathematical constant L provides a converging sequence of rational numbers $\frac{p_n}{q_n}$ (known as a *Diophantine approximation*). The formula can be characterized by *dynamical metrics* capturing properties such as its convergence rate. A recent paper [52] proposed using such metrics for formula discovery and clustering. Here we use the metrics of *convergence rate* and *irrationality measure*. The *convergence rate* is defined as:

$$r = \lim_{n \rightarrow \infty} \frac{1}{n} \log \left| L - \frac{p_n}{q_n} \right| \quad (4)$$

When examining the connection of two candidate formulas, the ratio of their r values can hint whether one is a transformation of a subsequence of the other (see Appendix E.5 for an example). The *irrationality measure* of $\frac{p_n}{q_n}$ is defined as the limit $\delta = \lim_{n \rightarrow \infty} \delta_n$, where

$$\delta_n = -1 - \frac{\log \left| L - \frac{p_n}{q_n} \right|}{\log |q_n|} \quad (5)$$

We found that two formulas sharing the same δ is the strongest indication of a possible relation, since δ is invariant under many transformations and choice of subsequences. Below, our UMAPS algorithm is used to derive and prove a relation once a pair of formulas share the same r and δ .

2.3 Conservative Matrix Fields (CMFs)

The CMF is the mathematical structure that generalizes formulas of a particular constant, originally found by generalizing PCFs [21], and later realized to be more general (Appendix G). To exemplify the

concept, we focus on the CMF of π . This CMF is 3D, i.e., consisting of three matrices M_x, M_y, M_z with rational function entries in the variables (x, y, z) , satisfying:

$$\begin{aligned} M_x(x, y, z)M_y(x+1, y, z) &= M_y(x, y, z)M_x(x, y+1, z) \\ M_x(x, y, z)M_z(x+1, y, z) &= M_z(x, y, z)M_x(x, y, z+1) \\ M_y(x, y, z)M_z(x, y+1, z) &= M_z(x, y, z)M_y(x, y, z+1) \end{aligned}$$

This property describes the path-independence of the transition between two points in a 3D lattice (lattice illustrated in Fig. 5b), in analogy to a conservative vector field. The CMF satisfies the properties of a discrete flat connection [11]. For an explanation of how formulas reside as directions within the CMF, see Appendix G.1. A notable feature of the CMF is that pairs of formulas found to be parallel trajectories with different initial points correspond to two matrices that are coboundary equivalent.

Many of the known π formulas will be shown to reside within a single CMF (details in Appendix G.2):

$$M_x = \begin{pmatrix} 1 & y \\ \frac{1}{x} & \frac{2x+y-2z+2}{x} \end{pmatrix} \quad M_y = \begin{pmatrix} 1 & x \\ \frac{1}{y} & \frac{x+2y-2z+2}{y} \end{pmatrix} \quad M_z = \begin{pmatrix} \frac{z(-x-y+z)}{(y-z)(x-z)} & \frac{zxy}{(y-z)(x-z)} \\ \frac{z}{(y-z)(x-z)} & \frac{-z^2}{(y-z)(x-z)} \end{pmatrix} \quad (6)$$

3 Methodology for symbolic unification of formulas

3.1 Harvesting: large-scale retrieval of formulas from the literature

The first challenge lies in the natural language processing of formulas. We analyze the L^AT_EX source code of 455,050 arXiv articles by combining regular expressions and LLMs, extracting all mathematical expressions, resulting in 278,242,506 strings. Filtering for expressions containing the π symbol retrieves 121,662 π -related equations. The widespread use of the symbol π in scientific literature means that most occurrences are unrelated to calculating the constant itself. To address this, and keeping in mind that there is a priori very little data on what successful formulas’ L^AT_EX looks like, each potential formula is classified as computing π or not, using GPT-4o mini [41] (chosen for its cost-effectiveness), reducing the number of candidates to 3367. Next, GPT-4o categorizes formulas by type: series, continued fraction, or neither—resulting in 1656 formula candidates.

3.2 Harvesting: extraction and validation

The extraction and validation stages rely on an LLM-code feedback loop that feeds a PSLQ algorithm. Each equation, represented as a L^AT_EX string, must then be parsed into a Computer Algebra System (CAS) for further manipulations (in our case, SymPy [38]). Automatically extracting algebraic forms from L^AT_EX strings is especially complicated due to varied L^AT_EX patterns, which are difficult

Harvesting scheme	Example
 455,050 $\hookrightarrow$ articles	Guillera, Jesús. "Bilateral sums related to Ramanujan-like series." <i>arXiv preprint arXiv:1610.04839</i> (2016).
(a)  scraping 278,242,506 $\hookrightarrow$ equations	Below, we show how to solve the problem for the Ramanujan-type series for $1/\pi$. Consider, for example the Ramanujan series $(21) \quad \sum_{n=0}^{\infty} (-1)^n \frac{\binom{3}{n} \binom{1}{n} \binom{1}{n}}{\binom{1}{n}} \frac{21460n + 1123}{882^{2n}} = \frac{3528}{\pi}.$
(b)  retrieval 121,662 $\hookrightarrow$ equations	$\text{"sum}_{n=0}^{\infty} (-1)^n \frac{\text{frac}(\text{frac}(12, n) \text{frac}(14, n)}{\text{frac}(34, n) \text{frac}(1, n^3)} \text{frac}(21460n + 1123) \{882^{2n}\}} = \text{frac}(3528) \{ \pi \}."$
(c)  computes π ?  series or continued fraction?	True series
(d)  extract 660 $\hookrightarrow$ formulas	term: $(-1)^n * \text{RisingFactorial}(1/2, n) * \text{RisingFactorial}(1/4, n) * \text{RisingFactorial}(3/4, n) / (\text{RisingFactorial}(1, n)^{**3} * (21460n + 1123) / 882^{** (2*n)})$ start: 0 variable: n
(e)  validate via PSLQ 385 $\hookrightarrow$ formulas	1122.99727845641348 $\approx$ 3528 / π
(f)  to recurrence 385 $\hookrightarrow$ formulas	$\begin{aligned} &(-14081/169923712 - (1946417n)/898359488 - (1368229n^2)/677699616 - (46871n^3)/556474968 \\ &\quad - n^4/777924) * f(n) + (-71386776899/847618568 - (183662894911n)/898359488 \\ &\quad - (1222951171699n^2)/6677699616 - (39244483773n^3)/556474968 - (777923n^4)/777924) * f[1 + n] \\ &\quad + (45366/5365 + (118659n)/5365 + (195080n^2)/18738 + (151393n^3)/21468 + n^4) * f[2 + n] = 0 \end{aligned}$

Figure 3: **Pipeline for automated harvesting of mathematical formulas (left), exemplified using one of the analyzed π formulas (right).** (a) Equations are scraped from papers on arXiv. (b) Regular expressions on the L^AT_EX strings retrieve series and continued fraction patterns that contain π as the only irrational number (see Appendix I.3). (c) Zero-shot classification using OpenAI’s GPT-4o mini identifies formulas calculating the constant π . Then, OpenAI’s GPT-4o identifies the formula type (series, continued fraction, or neither). (d) Extraction of the series’ summand or the continued fraction’s partial numerator and partial denominator, using GPT-4o. The formula is then converted to code. (e) Formulas are computed and validated using the integer relation finder algorithm PSLQ. (f) The formulas are converted to canonical recurrences using RISC’s tool for fitting recurrences [33].

to systematically convert to executable code using a predefined logic. LLMs help us overcome these obstacles by processing text contextually and attending to relevant parts of the text, solving the natural language processing task that may have required elaborate rules [14, 47]. Specifically, we use OpenAI’s GPT-4o to translate relevant L^AT_EX into executable mathematical code [22, 43, 61] (see Appendix I for the exact prompts used). To correct for (common) mistakes in the LLM-generated formula code, we apply an LLM-code feedback loop for code validation: errors are sent back to the LLM along with the faulty code to correct it, up to three times (see Appendix I.6.3).

We validate that each extracted formula computes the constant π by running the formula code to get a numerical approximation and then applying PSLQ, an integer relation algorithm [26]. Limit values are not extracted directly from the L^AT_EX string for validation, since we found that GPT-4o got them wrong in some cases (see Table 14). Instead, the PSLQ approach fixes these critical GPT mistakes and reproduces the intended formulas. Out of the 660 candidates, 385 were validated as π formulas and passed on for canonicalization (details in Appendix I.5).

3.3 Clustering: using the canonical form

The first unification step is converting each formula to its canonical form: the simplest linear recurrence with polynomial coefficients (Appendix E.4.1). Automated algebraic capabilities are unpredictable in solving such tasks. Thus, we use a computational method for converting the formulas to polynomial recurrences: a Mathematica package by RISC [33] that fits polynomial-coefficient linear recurrences to each sequence of rational numbers. The resulting recurrences are validated numerically and passed to a Maple package to guarantee order minimality [57, 60], thus finding the provably minimal polynomial recurrence. Out of the 385 validated formulas (Section 3.1), 380 are found to have representations as order-2 recurrences, and 5 as order-3 recurrences, which can also be addressed as we show in Appendix B.6 and Appendix C.3.

The same canonical form captures a wide range of formulas, continued fractions and infinite sums. Thus, the conversion to canonical forms automatically unifies different formulas, yielding 149 different order-2 canonical forms and 4 order-3 canonical forms for π , 153 in total, from 385 formulas (selected examples in Table 1).

Table 1: **Canonical form representation.** Converting formulas to their canonical forms shows equivalence of different-looking expressions (e.g. 1,2), leaving the less-trivial connections for the later steps of the algorithm. Additional details in Appendix C.4.

	Formula	Value	arXiv source	Canonical form (CF)	CF value	Initial conditions
1	$\sum_{n=0}^{\infty} \frac{n!}{\prod_{i=1}^n (2i+1)}$	$\frac{\pi}{2}$	1806.03346	PCF($3n+1, n(1-2n)$)	$\frac{2}{\pi}$	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$
2	$\sum_{n=1}^{\infty} \frac{2^n}{n \binom{2n}{n}}$	$\frac{\pi}{2}$	2010.05610	PCF($3n+1, n(1-2n)$)	$\frac{2}{\pi}$	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$
3	$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$	$\frac{\pi}{4}$	2404.15210	PCF($2, (2n-1)^2$)	$1 + \frac{4}{\pi}$	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$
4	$\sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n(n+1)(2n+1)}$	$\pi - 3$	2206.07174	PCF($6, (2n+1)^2$)	$\frac{1}{\pi-3}$	$\begin{bmatrix} 0 & 1 \\ 1 & 6 \end{bmatrix}$
5	$\sum_{n=1}^{\infty} \frac{4^n(12n-5)}{(2n-1)\binom{4n}{2n}}$	$\frac{3\pi+4}{2}$	2204.08275	*	$\frac{-42\pi-196}{3\pi+4}$	$\begin{bmatrix} 0 & 70 \\ -1 & 15 \end{bmatrix}$
* PCF($240n^3 + 164n^2 - 54n - 29, -9216n^6 + 12288n^5 + 11264n^4 - 15520n^3 - 764n^2 + 3802n - 714$)						

3.4 Clustering: using the dynamical metrics

The clustering stage is a heuristic to guide which formulas should be attempted to be proven equal using UMAPS. Formulas with the same metrics are likely to be related to the same constant [52]. The metrics also indicate a more intricate connection, enabling the unification of formulas in a systematic way that proves an analytical transformation between them. Canonical-form formulas are first compared to each other using the irrationality measure δ (Fig. 4a), which is the most reliable indicator for a potential equivalence. Every new formula is first evaluated relative to directions in the CMF corresponding to recurrences with the same δ . This search can be improved by using gradient descent on the direction parameters, because δ is found to be continuous [21].

We found that δ is not sufficient to imply equivalence, and thus we complement it using the ratio of convergence rates $r_A : r_B$. Canonical form A is *folded* (Appendix E.5) by r_B and canonical form B is folded by r_A (Fig. 4b), making them converge at the same rate. The next step is finding their precise algebraic relation using UMAPS.

3.5 Unification: using the UMAPS algorithm for coboundary equivalence

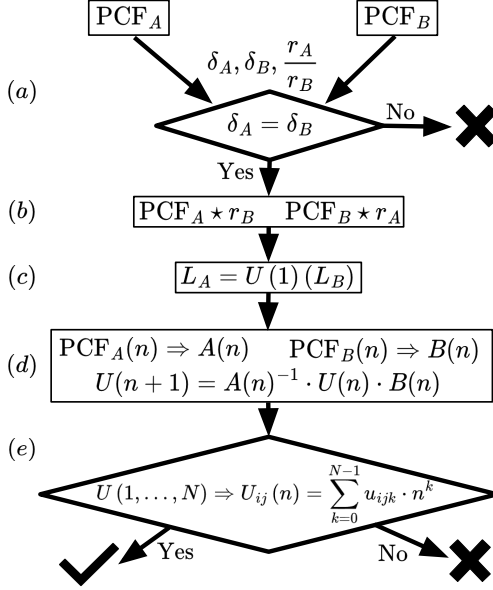


Figure 4: The matching algorithm: connecting polynomial linear recurrences. This algorithm is demonstrated here for polynomial continued fractions (PCFs) but can be generalized to any linear polynomial recurrence. (a) Compute the dynamical metrics [52] for the two PCFs (irrationality measures δ_A, δ_B and the convergence rates ratio r_A/r_B). The δ metrics are used to identify possible connections, as only if $\delta_A = \delta_B$, the PCFs can be related via coboundary (in practice, we test for them to be within 0.06 of each other). (b) Fold PCF_A by r_B and PCF_B by r_A (Appendix E.5). **UMAPS (c)-(e):** (c) Solve for a general Möbius transform (a 2×2 matrix $U(1)$) that once applied to the limit of PCF_B equates it to the limit of PCF_A . (d) Representing the PCFs in matrix form ($A(n)$ and $B(n)$), propagate the coboundary matrix via the relation $U(n+1) = A(n)^{-1} \cdot U(n) \cdot B(n)$ up to $U(N)$ ($N = 40$ was sufficient for our runs, see Appendix D). (e) Assume the general form of $U(n)$ to have rational-function entries with polynomial degree up to $\lfloor \frac{N-1}{2} \rfloor$ and solve for their coefficients using normalized $U(1, \dots, N)$. If such a solution is found and validated, the PCFs are coboundary-related. See Appendix C for more details.

Our algorithm for unification via mapping across symbolic structures (UMAPS) relies on the established concept of coboundary equivalence (Appendix E.4), however, no specialized coboundary solver existed prior to this work.

$A(n), B(n) \in \text{PGL}_m(\mathbb{Q}(n))$ are *coboundary equivalent* if there exist a matrix $U(n)$ such that

$$A(n) \cdot U(n+1) = U(n) \cdot B(n) \quad (7)$$

This definition carries to recurrences when their companion matrices (Eq. (1)) are coboundary equivalent (Fig. 5a,d) and then: $(\prod_{i=1}^n A(i)) \cdot U(n+1) = U(1) \cdot (\prod_{i=1}^n B(i))$.

Since any matrix with rational function coefficients can be scaled to have polynomial coefficients, we can write that $A(n), B(n) \in \text{GL}_m(\mathbb{Q}[n])$ are *coboundary equivalent* if there exist a matrix $U(n) \in \text{GL}_m(\mathbb{Q}[n])$ and polynomials $p_A(n), p_B(n) \in \mathbb{Q}[n]$ such that

$$p_A(n) \cdot A(n) \cdot U(n+1) = p_B(n) \cdot U(n) \cdot B(n) \quad (8)$$

Finding a coboundary between two polynomial matrices is inherently a non-linear problem due to the product of unknown polynomials p_A and p_B with unknown coboundary matrix U . Moreover, the degree of each polynomial is not known. Despite the non-linearity, we found a coboundary solver algorithm for general order m (Appendix C.3).

UMAPS finds the solution without solving non-linear equations, instead leveraging the recurrence limits to compute a sequence of empirical coboundary matrices, whose elements are fitted to rational functions [53]. The algorithm relies on the following lemma:

Lemma 1. (A necessary condition on the coboundary equivalence matrix.) Let $L_A = \lim_{n \rightarrow \infty} PCF(a(n), b(n))$ and $L_B = \lim_{n \rightarrow \infty} PCF(c(n), d(n))$ be converging PCFs with associated companion matrices $A(n), B(n) \in \text{PGL}_2(\mathbb{Q}(n))$. If $A(n)$ is coboundary to $B(n)$, then L_A and L_B are related through a rational Möbius transformation. Moreover, if $U(n)$ is the coboundary matrix, then $L_A = U(1)(L_B)$ ($U(1)$ applied to L_B as a Möbius transformation).

A proof and generalization to higher-order recurrences (Lemma 4), as well as a proof of the uniqueness of the coboundary matrix (Lemma 5), are detailed in Appendix F. These combine to show that UMAPS is sufficient to solve for the coboundary matrix, as stated in Corollary 1 (proof in Appendix C.3).

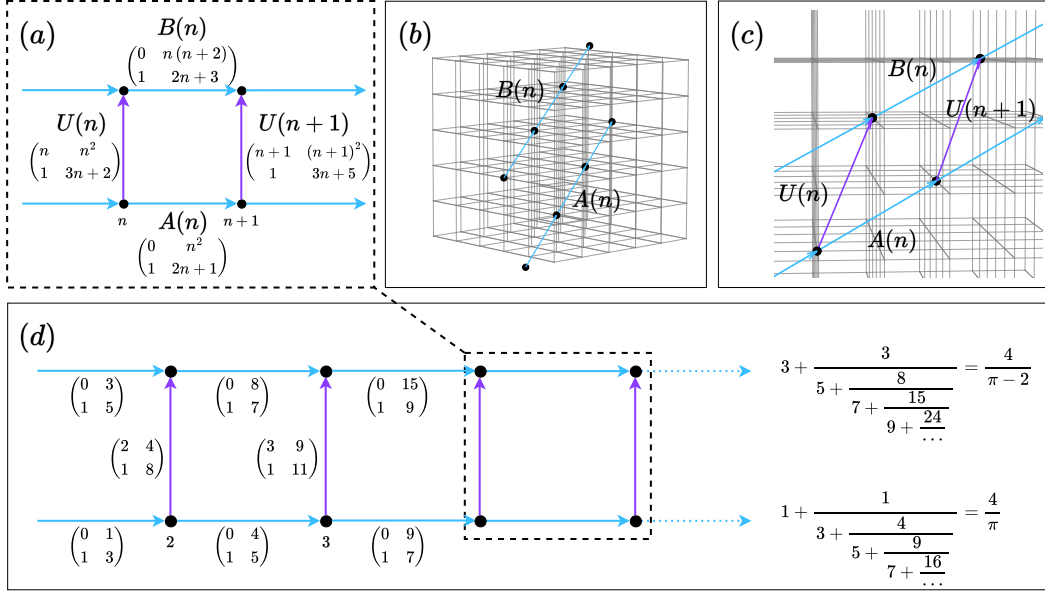


Figure 5: **Coboundary equivalence:** the mathematical framework connecting different formulas once cast into their canonical forms. (a) The coboundary condition $A(n) \cdot U(n+1) = U(n) \cdot B(n)$ recasts formulas as (b,c) parallel trajectories in a CMF. (d) Example of two coboundary-equivalent formulas, presenting their coboundary matrices and limits, which constitute proof of a novel equivalence.

Corollary 1. (Sufficiency of UMAPS.) *If a coboundary matrix exists for two matrices and every rational-function entry of the coboundary matrix has polynomials of degree at most d , then running UMAPS with $N \geq 2d + 1$ suffices to recover the coboundary matrix.*

Fig. 4 summarizes the flow of matching two canonical form formulas. Using this method, we find that formulas 1,2 and 5 from Table 1 are equivalent and that formulas 3,4 are also equivalent. Refer to Appendix B.1 for descriptions of how the algorithm is applied to these formulas, and refer to Appendix C for a listing of the algorithms involved. A study of the algorithms’ sensitivity to hyperparameters is provided in Appendix D. The same procedure is applied to each canonical form formula, measuring its δ value and relying on its continuity as a function of direction in the CMF to locate worthy directions that yield potential formula pairs for the coboundary algorithm. The matching algorithm is then applied between formulas and representative recurrences from the CMF. Finding a match between a formula and a CMF representative proves the formula is generated by the CMF. The full list of results is provided in Appendix J. Selected results for π are detailed in Section 5.

4 Benchmarking

4.1 Comparison to other methods for symbolic unification

Our work is the first to address the problem of symbolic unification at scale, thus there are no standard benchmarks for performance comparison. Leading LLMs are generally unable to address the full challenge. As an example, we compare our equivalence detection and proving capabilities to those of LLMs: We tasked 2 leading LLMs—GPT-4o and Gemini 2.5 Pro Preview—with identifying and proving 10 formula-pair equivalences proven by our algorithm (Table 2). The formulas are chosen such that each pair has equal dynamical metrics (r, δ) after folding, which is the simpler situation to prove (parallel trajectories in the CMF). Even with these simpler tasks, the LLMs exhibit only limited success. We did not find cases in which any LLM succeeded in finding relations between a pair of formulas without equal dynamical metrics.

4.2 Comparison of LLM model performance

We utilize LLMs for classification and extraction in two different ways. Table 3 compares the performance of three choices for the extractor LLM, which we found to be the more sensitive choice, as it is used for the more advanced LLM-code feedback loop. A ground truth is established by merging the validated formulas (Section 3.2) found by the three compared LLMs.

Table 2: LLM performance when detecting and proving equivalence in a random set of 10 formula pairs of equal dynamical parameters (Appendix H). All LLM proofs were validated manually.

LLM	Successful detections	Correct proofs
GPT-4o	1/10	2/10
Gemini 2.5 Pro Preview	8/10	5/10

Table 3: Performance of different extractor LLM choices in terms of successfully harvested formulas. The LLM errors are split to “faulty code” for code that did not run, and “symbolic mistake” for incorrect identification of some of the formula constituents like continued fraction polynomials. The bold row marks the choice of LLMs used for all the rest of the results in this paper.

LLM classifier	LLM extractor	Successful extractions	Faulty code	Symbolic mistake
GPT-4o mini	GPT-4o	289 (97.6%)	2 (0.7%)	5 (1.7%)
GPT-4o mini	Claude 3.7 Sonnet	266 (89.9%)	21 (7.1%)	9 (3.0%)
GPT-4o mini	GPT-4o mini	206 (69.6%)	70 (23.6%)	20 (6.8%)

5 Results

5.1 Example equivalences among famous formulas

Our automated system proves previously unknown equivalences between formulas. Among the formulas connected are famous examples, such as one of Ramanujan’s 1914 formulas, as well as Lord Brouncker’s, Euler’s, and Gauss’s PCFs from the 17th, 18th, and 19th centuries [23, 28, 42]. For example, the following series found by Ramanujan in 1914 [48],

$$\frac{4}{\pi} = \sum_{k=0}^{\infty} (-1)^k \frac{\left(\frac{1}{2}\right)_k \left(\frac{1}{4}\right)_k \left(\frac{3}{4}\right)_k}{(1)_k^3} \cdot (1123 + 21460k) \cdot \left(\frac{1}{882}\right)^{2k+1} \quad (9)$$

was proven equivalent (Appendix B.4) to a newer series from a paper published in 2020 [54]:

$$\frac{341446000}{\pi} = \sum_{k=0}^{\infty} \frac{\binom{2k}{k}^2 \binom{4k}{2k}}{(k+1)(2k-1)(4k-1)(-2^{10}21^4)^k} \cdot (1424799848k^2 + 1533506502k + 1086885699) \quad (10)$$

This equivalence demonstrates how two previously distinct mathematical expressions, discovered over a century apart, are now proven to be equivalent by an automated process.

Fig. 5d proves the equivalence of another pair of famous formulas: (1) PCF($2n+3, n(n+2)$), one of the first computer-discovered π formulas from 2021 [46]. (2) PCF($2n+1, n^2$), published by Gauss in 1813 [28] and provided at the time an especially efficient way to calculate digits of π .

5.2 Formulas unified by a Conservative Matrix Field (CMF)

The CMF of π , Eq. (6), captures most of the harvested formulas (Table 4), with selected examples presented graphically in Fig. 6 along with their corresponding trajectories.

Table 4: **Summary of unification results**, among all validated formulas (left columns) and among the canonical forms (right columns). Formulas are harvested from 140 arXiv papers (Table 15), of which 137/140 (98%) have at least one formula proved connected by UMAPS and 70/140 (50%) have a formula residing in the same CMF.

Found relation	Same CMF	Found relation (canonical)	Same CMF (canonical)
360/385 (94%)	166/385 (43%)	136/153 (89%)	81/153 (53%)

The full list of canonical forms captured by the CMF appears in Table 16. Improvements in UMAPS are likely to connect additional formulas (Table 17) to the same CMF.

5.3 Unification of formulas beyond π

Going beyond π , we automatically identified equivalent formulas for e , $\zeta(3)$, and Catalan’s constant—demonstrating the generality of the approach. Consider these two formulas for Apéry’s constant, the Riemann zeta function value $\zeta(3)$:

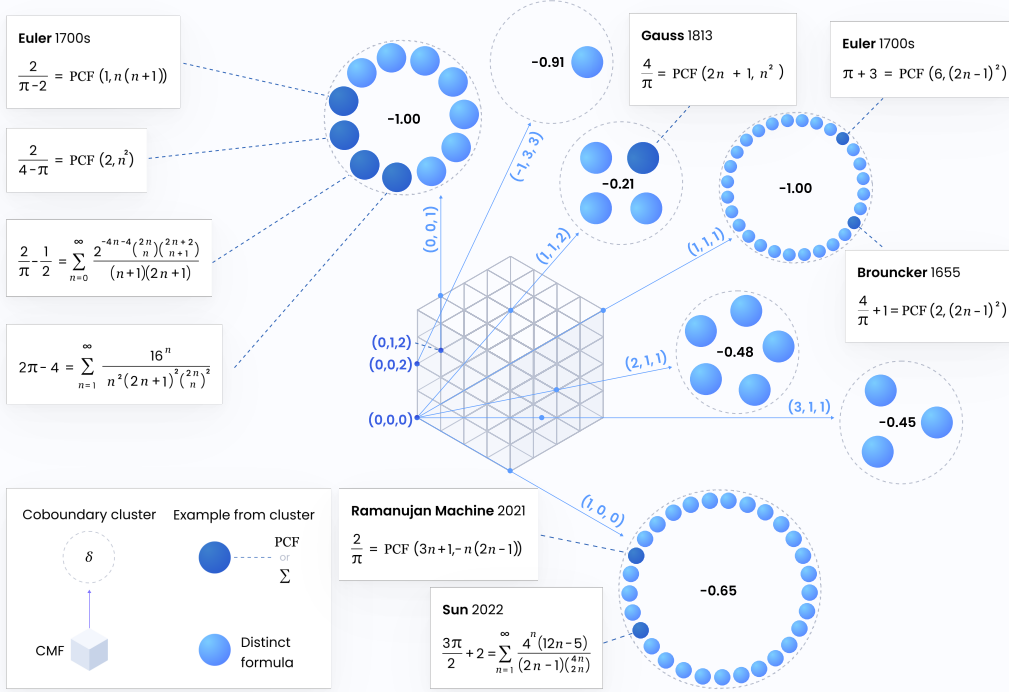


Figure 6: **Formula unification by a Conservative Matrix Field (CMF).** Numerous π formulas harvested from the literature are automatically arranged as trajectories in a 3D CMF. These formulas include famous ones by Gauss, Euler, and Lord Brouncker. The full list of unified formulas and their canonical forms is available in Table 16. Each cluster (large dashed circles) denotes formulas connected by coboundary, representing parallel trajectories or overlapping trajectories. The number at each cluster center is the δ of all formulas in that cluster. Arrows show trajectory directions. Note that multiple formula clusters can have the same δ value without being coboundary, showing that sharing δ is necessary but not sufficient for formulas being coboundary-related.

$$\zeta(3) = \sum_{n=1}^{\infty} \frac{1}{n^3} \qquad \frac{5}{4} - \zeta(3) = \sum_{n=2}^{\infty} \frac{1}{n^3(n^2-1)} \quad (11)$$

The second formula [36] has faster convergence compared to the classical definition of $\zeta(3)$, though both converge polynomially. Our automatic procedure proves their equivalence by the coboundary transform and unifies them in the $\zeta(3)$ CMF (detailed in Appendix B.2).

As another example, the following two PCFs for Catalan's constant [13] are also proven equivalent by UMUPS (Appendix B.5).

$$\frac{1}{2-2G} = 3 + \frac{2^2}{1 + \frac{2^2}{3 + \frac{4^2}{1 + \frac{4^2}{\dots}}}} \qquad \frac{1}{2G-1} = \frac{1}{2} + \frac{1^2}{\frac{1}{2} + \frac{1 \cdot 2}{2^2 + \frac{2 \cdot 3}{\frac{1}{2} + \dots}}} \quad (12)$$

A natural next step is to conduct exhaustive searches for other well-known constants, and fundamental mathematical structures in fields such as physics and computer science. See Appendix B.3 for e examples.

5.4 Formulas generated via a Conservative Matrix Field (CMF)

A sample of 1693 distinct π formulas was generated from the π -CMF (per Appendix C.5). The CMF permits a new method of comparing between formulas, using the *normalized convergence rate*, defined $r/\ell^1(t)$, where r is the convergence rate from Eq. (4), t is a trajectory and ℓ^1 is the ℓ^1 -norm. 57 of the formulas generated in our runs shared the best normalized r of 1.76, such as the formula

$$\frac{10}{4-\pi} = 12 + \frac{-81}{238 + \frac{-6500}{968 + \frac{-67473}{n^2(-64n^4 - 96n^3 + 12n^2 + 52n + 15)} + \frac{-67473}{48n^3 + 108n^2 + 70n + 12} + \dots}}$$

arising from trajectory $(-1, -1, 0)$. By comparison, the best pre-existing formula unified by our CMF has normalized convergence of 0.88 (the $(1, 1, 2)$ direction, see Table 16). Details in Appendix G.6.

6 Discussion

6.1 Limitations

Currently, the harvesting step relies on the LLM’s ability to interpret and contextualize mathematical $\text{\LaTeX}$ strings. This step likely introduces data loss and false negatives in formula classification. Improvements in prompt engineering and in validation techniques will enhance the robustness of this LLM use. As more advanced LLMs become available, this step will become increasingly reliable.

Formulas often include additional symbols other than summation indices, like variables defined in the text surrounding the formula, which should be extracted and substituted into formula evaluation. We made several such substitutions manually to test the rest of the pipeline for these special cases. Future improvements of the unification pipeline can address this limitation by more advanced use of LLMs and automated validation.

Most formulas analyzed in this work are series or continued fractions. However, UMAPS and all the other steps in our harvesting and clustering processes are applicable more broadly (to any formula that generates a sequence of rational approximations for a given constant, e.g., deeper recurrences). Expanding the system to accommodate additional cases is a promising direction for future work.

The same unification pipeline shown here could apply to the vast family of constants derived from D-finite functions by finding their corresponding CMFs [58].

6.2 Outlook

Increasing the dimension and rank of the π -CMF, along with further improvements to UMAPS [2], is likely to yield a higher percentage of unified formulas in the near future. A planned future study will employ the CMF to systematically search for fast-converging and irrationality-proving formulas.

Looking forward, the same approach of collection, analysis, and organization of mathematical knowledge could help establish rigorous connections between different branches of mathematics. The methodology presented in this work could help develop more general frameworks for identifying connections between different scientific theories through their mathematical representations. As the volume of information grows at an accelerating pace, finding automated ways to unify knowledge will become increasingly essential, especially with the goal of providing more intuitive understanding on complex concepts.

Pairing LLMs with pre-existing and novel tools for symbolic and numerical mathematics enabled the automated discoveries in this paper. We believe this LLM–tool integration scheme will continue to advance AI for mathematics and science in the coming years.

- [19] Saber Elaydi. *An Introduction to Difference Equations*. Undergraduate Texts in Mathematics. Springer Nature, New York, third edition, 2006.
- [20] Rotem Elimelech, Ofir David, Carlos De la Cruz Mengual, Rotem Kalisch, Wolfgang Berndt, Michael Shalyt, Mark Silberstein, Yaron Hadad, and Ido Kaminer. Algorithm-assisted discovery of an intrinsic order among mathematical constants. *arXiv preprint arXiv:2308.11829*, 2023.
- [21] Rotem Elimelech, Ofir David, Carlos De la Cruz Mengual, Rotem Kalisch, Wolfgang Berndt, Michael Shalyt, Mark Silberstein, Yaron Hadad, and Ido Kaminer. Algorithm-assisted discovery of an intrinsic order among mathematical constants. *Proceedings of the National Academy of Sciences (PNAS)*, 121(25):e2321440121, 2024.
- [22] Hasan Ferit Eniser, Hanliang Zhang, Cristina David, Meng Wang, Maria Christakis, Brandon Paulsen, Joey Dodds, and Daniel Kroening. Towards translating real-world code with llms: A study of translating to rust. *arXiv preprint arXiv:2405.11514*, 2024.
- [23] Leonhard Euler. *Introductio in analysin infinitorum*, volume 2. 1748.
- [24] Siemion Fajtlowicz. On conjectures of graffiti. In J. Akiyama, Y. Egawa, and H. Enomoto, editors, *Graph Theory and Applications*, volume 38 of *Annals of Discrete Mathematics*, pages 113–118. Elsevier, 1988.
- [25] Alhussein Fawzi et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022.
- [26] Helaman R. P. Ferguson, David H. Bailey, and Paul Kutler. *A Polynomial Time, Numerically Stable Integer Relation Algorithm*. Ames Research Center, 1998.
- [27] Luyu Gao et al. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR, 23–29 Jul 2023.
- [28] Carl Friedrich Gauss. *Werke*, vol. 3, 1813.
- [29] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *International Conference on Learning Representations (ICLR)*, 2024.
- [30] Jesus Guillera. History of the formulas and algorithms for pi. *arXiv preprint arXiv:0807.0872*, 2008.
- [31] Milad Hashemi et al. Can transformers do enumerative geometry? In *International Conference on Learning Representations (ICLR)*, 2025.
- [32] Pierre-Alexandre Kamienny, Stéphane d’Ascoli, Guillaume Lample, and François Charton. End-to-end symbolic regression with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 10269–10281. Curran Associates, Inc., 2022.
- [33] Manuel Kauers and Christoph Koutschan. Guessing with little data. In *Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation, ISSAC ’22*, page 83–90. ACM, July 2022.
- [34] Walter G. Kelley and Allan C. Peterson. *Difference equations: An introduction with applications*. Harcourt/Academic Press, San Diego, CA, second edition, 2000.
- [35] John R. Koza. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4:87–112, 1994.
- [36] Ernst Eduard Kummer. *Eine neue Methode, die numerischen Summen langsam convergirender Reihen zu berechnen*. Walter de Gruyter, Berlin/New York, 1837.
- [37] L. J. Lange. An elegant continued fraction for π . *American Mathematical Monthly*, 106: 456–458, 1999.